

The aim of line (1) is to obtain the data entered by the user and store it in a variable, namely, 'data'. In line (2), we aim to map this data against the Student model via our StudentSerializer and check for the presence of 'student_name', 'roll_n' and 'detailed_data' fields, while also validating the type of values entered for the first two fields. If the 'serializer' is valid, i.e., if none of the three fields are missing and there is no objection to the type of values entered for the former two fields, we check the data within the 'detailed_data' JSONField. After assigning the manually-written schema to 'myschema' in line (4), line (5) passes the schema to DraftValidator and the same is assigned to 'v'. The 'v.iter_errors(data["detailed_data"])' in line (6) intends to point out any validation errors, should they exist, such as missing fields, wrong value entered for fields, etc. We simply append these errors in a list as 'elements' and check the length of the list. If the length is 0, it means the data entered in the JSONField abides by the schema and validation is successful. If not, the validation errors are displayed as a list, serving as 'value' to the 'error' key mentioned in line (7).

In Figure 5, we can see that values for all the fields are exactly of the types written in the schema. Additionally, there are no missing fields, which results in the particular entry being saved without any validation errors. Let's try making a few changes to see how the validation errors are shown. Let's not include the first 'professor' key-value pair in 'semester_2_subjects'. Do the same for the second 'subject_name' within 'semester_1_subjects'. Let's write an integer type value for 'department' within 'teacher_information' and a wrong pattern for 'date_of_joining'.

We have covered the various types of validation errors. This method requires us to write out the schema manually, but what if you didn't want to do so and what if it could be done for you instead? This is where JSON models come into play.

Using JSON models

To generate the previous schema, the following additions have to be made in *models.py*.

models.py

```
from jsonmodels import models as m1, fields

class teacherField(m1.Base):
    name=fields.StringField(required=True)
    date_of_birth=fields.DateField(required=True)
    department=fields.StringField(required=True)
    joining_date=fields.DateField(required=True)
class sem1Field(m1.Base):
    subject_name=fields.StringField(required=True)
    professor=fields.StringField(required=True)
    score=fields.IntField(required=True)
class sem2Field(m1.Base):
    subject_name=fields.StringField(required=True)
    professor=fields.StringField(required=True)
    score=fields.IntField(required=True)
class detailed_data_field(m1.Base):
    date_of_joining=fields.DateField(required=True)
    mother_name=fields.StringField(required=True)
    father_name=fields.StringField(required=True)
```

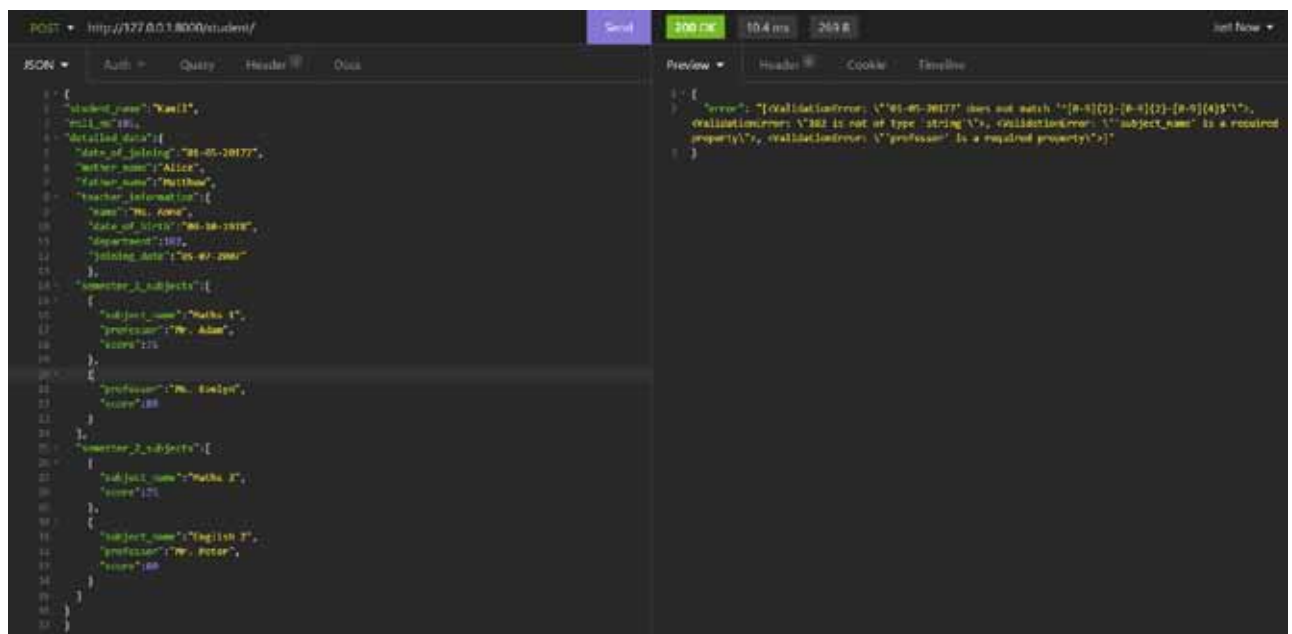


Figure 6: Display of list of validation errors